CATEGORY C

Abstraction, Interface & Class vs Interface

Complete Java Coding Assignment – Answer Key

Problems covered

1. Movie Ticket Counter
2. Parcel Shipping Desk
3. College Fee Counter
4. City Cab Fare Meter
5. Home Appliance Energy Report

Includes complete standalone Java programs, OOP design explanations, and the sample input/output specified in the supplied assignment.

Prepared from the supplied 5-page Category C assignment.

How to use these solutions

Save each program in a file whose name matches its public class name. Compile and run one program at a time. Each program reads the number of records first, following the input format in the assignment.

OOP concepts used

Abstraction: an abstract class defines common behaviour and leaves type-specific details to subclasses.
Interface: a capability contract such as Insurable, BusUser, NightService, or SaverMode. Only classes supporting that capability implement it.
Polymorphism: a parent-class reference can refer to different subclass objects, and the overridden method for the actual object is executed.

The sample outputs below reproduce the expected outputs printed in the supplied assignment. The code uses the same business rules and formatting.

Problem 1 – Movie Ticket Counter

OOP design / explanation

An abstract Ticket class stores the convenience fee once and provides the common amount calculation. Each seat subclass supplies its own price. Because Ticket is abstract, a generic ticket without a seat type cannot be instantiated.

Complete Java solution

```
import java.util.Locale;
import java.util.Scanner;

abstract class Ticket {
    private static final double CONVENIENCE_FEE = 20.0;
    protected abstract double getSeatPrice();

    public final double amount(int count) {
        return count * (getSeatPrice() + CONVENIENCE_FEE);
    }
}

class RegularTicket extends Ticket {
    protected double getSeatPrice() { return 150.0; }
}
class PremiumTicket extends Ticket {
    protected double getSeatPrice() { return 250.0; }
}
class ReclinerTicket extends Ticket {
    protected double getSeatPrice() { return 400.0; }
}

public class MovieTicketCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        double total = 0.0;

        for (int i = 0; i < n; i++) {
            String seat = sc.next().toUpperCase(Locale.ROOT);
            int count = sc.nextInt();
            Ticket ticket;

            if (seat.equals("REGULAR")) {
                ticket = new RegularTicket();
            } else if (seat.equals("PREMIUM")) {
                ticket = new PremiumTicket();
            } else if (seat.equals("RECLINER")) {
                ticket = new ReclinerTicket();
            } else {
                throw new IllegalArgumentException("Unknown seat type: " + seat);
            }

            double amount = ticket.amount(count);
            total += amount;
            System.out.printf(Locale.US, "%s: %.2f%n", seat, amount);
        }
        System.out.printf(Locale.US, "Total: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
3
REGULAR 3
PREMIUM 2
RECLINER 1
```

Expected output

```
REGULAR: 510.00
PREMIUM: 540.00
RECLINER: 420.00
Total: 1470.00
```

Problem 2 – Parcel Shipping Desk

OOP design / explanation

Parcel is an abstract parent for the common shipping-charge contract. Insurable is a separate capability interface. Express and fragile parcels implement it; standard parcels do not, so standard insurance remains 0.00.

Complete Java solution

```
import java.util.Locale;
import java.util.Scanner;

interface Insurable {
    double calculateInsurance(double declaredValue);
}

abstract class Parcel {
    protected final double weightKg;
    public Parcel(double weightKg) { this.weightKg = weightKg; }
    public abstract double calculateCharge();

    public final double calculateTotal(double insurance) {
        return calculateCharge() + insurance;
    }
}

class StandardParcel extends Parcel {
    public StandardParcel(double weightKg) { super(weightKg); }
    public double calculateCharge() { return 40.0 + 10.0 * weightKg; }
}
class ExpressParcel extends Parcel implements Insurable {
    public ExpressParcel(double weightKg) { super(weightKg); }
    public double calculateCharge() { return 80.0 + 15.0 * weightKg; }
    public double calculateInsurance(double value) { return value * 0.02; }
}
class FragileParcel extends Parcel implements Insurable {
    public FragileParcel(double weightKg) { super(weightKg); }
    public double calculateCharge() {
        return 40.0 + 10.0 * weightKg + 50.0;
    }
    public double calculateInsurance(double value) { return value * 0.02; }
}

public class ParcelShippingDesk {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        double grandTotal = 0.0;

        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase(Locale.ROOT);

            double weight = sc.nextDouble();
            double declaredValue = sc.nextDouble();
            Parcel parcel;

            if (type.equals("STANDARD")) {
                parcel = new StandardParcel(weight);
            } else if (type.equals("EXPRESS")) {
                parcel = new ExpressParcel(weight);
            } else if (type.equals("FRAGILE")) {
                parcel = new FragileParcel(weight);
            } else {
                throw new IllegalArgumentException("Unknown parcel type: " + type);
            }

            double charge = parcel.calculateCharge();
            double insurance = 0.0;
            if (parcel instanceof Insurable) {
                insurance = ((Insurable) parcel)
                        .calculateInsurance(declaredValue);
            }
            double parcelTotal = parcel.calculateTotal(insurance);
            grandTotal += parcelTotal;

            System.out.printf(Locale.US,
                    "%s: Charge=%.2f Insurance=%.2f Total=%.2f%n",
                    type, charge, insurance, parcelTotal);
        }
        System.out.printf(Locale.US, "Grand Total: %.2f%n", grandTotal);
        sc.close();
    }
}
```

Sample input

```
3
STANDARD 3 500
EXPRESS 2 1000
FRAGILE 4 2000
```

Expected output

```
STANDARD: Charge=70.00 Insurance=0.00 Total=70.00
```

```
EXPRESS: Charge=110.00 Insurance=20.00 Total=130.00
FRAGILE: Charge=130.00 Insurance=40.00 Total=170.00
Grand Total: 370.00
```

Problem 3 – College Fee Counter

OOP design / explanation

Student is an abstract class for common student data and total-fee calculation. BusUser is a capability interface implemented by day scholars and scholarship students only. Hostellers add the hostel fee and do not pay transport.

Complete Java solution

```
import java.util.Locale;
import java.util.Scanner;

interface BusUser {
    double getTransportFee();
}

abstract class Student {
    protected final String name;
    public Student(String name) { this.name = name; }
    protected abstract double getTuition();
    protected double getOtherFees() { return 0.0; }

    public final double calculateTotalFee() {
        double transport = 0.0;
        if (this instanceof BusUser) {
            transport = ((BusUser) this).getTransportFee();
        }
        return getTuition() + getOtherFees() + transport;
    }
}

class DayScholar extends Student implements BusUser {
    public DayScholar(String name) { super(name); }
    protected double getTuition() { return 40000.0; }
    public double getTransportFee() { return 12000.0; }
}
class Hosteller extends Student {
    public Hosteller(String name) { super(name); }
    protected double getTuition() { return 40000.0; }
    protected double getOtherFees() { return 60000.0; }
}
class ScholarshipStudent extends Student implements BusUser {
    public ScholarshipStudent(String name) { super(name); }
    protected double getTuition() { return 20000.0; }
    public double getTransportFee() { return 12000.0; }
}

public class CollegeFeeCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        double totalCollected = 0.0;

        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase(Locale.ROOT);
            String name = sc.next();
            Student student;

            if (type.equals("DAY_SCHOLAR")) {
                student = new DayScholar(name);
            } else if (type.equals("HOSTELLER")) {
                student = new Hosteller(name);
            } else if (type.equals("SCHOLAR")) {
                student = new ScholarshipStudent(name);
            } else {
                throw new IllegalArgumentException("Unknown student type: " + type);
            }

            double fee = student.calculateTotalFee();
            totalCollected += fee;
            System.out.printf(Locale.US, "%s: %.2f%n", name, fee);
        }
        System.out.printf(Locale.US, "Total Collected: %.2f%n", totalCollected);
        sc.close();
    }
}
```

Sample input

```
3
DAY_SCHOLAR Asha
HOSTELLER Ravi
SCHOLAR Neha
```

Expected output

```
Asha: 52000.00
Ravi: 100000.00
Neha: 32000.00
Total Collected: 184000.00
```

Problem 4 – City Cab Fare Meter

OOP design / explanation

Cab centralizes the minimum fare rule in one final method. NightService is a capability interface implemented by sedans and SUVs, but not mini cabs. A MINI NIGHT request is therefore rejected and is not added to the total.

Complete Java solution

```
import java.util.Locale;
import java.util.Scanner;

interface NightService {
    double addNightSurcharge(double fare);
}

abstract class Cab {
    protected abstract double getRatePerKm();

    public final double calculateDayFare(double km) {
        return Math.max(100.0, km * getRatePerKm());
    }
}

class MiniCab extends Cab {
    protected double getRatePerKm() { return 10.0; }
}
class SedanCab extends Cab implements NightService {
    protected double getRatePerKm() { return 14.0; }
    public double addNightSurcharge(double fare) { return fare * 1.20; }
}
class SUVCab extends Cab implements NightService {
    protected double getRatePerKm() { return 18.0; }
    public double addNightSurcharge(double fare) { return fare * 1.20; }
}

public class CityCabFareMeter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        double total = 0.0;

        for (int i = 0; i < n; i++) {
            String type = sc.next().toUpperCase(Locale.ROOT);
            double km = sc.nextDouble();
            String time = sc.next().toUpperCase(Locale.ROOT);
            Cab cab;

            if (type.equals("MINI")) {
                cab = new MiniCab();
            } else if (type.equals("SEDAN")) {

                cab = new SedanCab();
            } else if (type.equals("SUV")) {
                cab = new SUVCab();
            } else {
                throw new IllegalArgumentException("Unknown cab type: " + type);
            }

            if (time.equals("NIGHT") && !(cab instanceof NightService)) {
                System.out.println(type + ": night service not available");
                continue;
            }

            double fare = cab.calculateDayFare(km);
            if (time.equals("NIGHT")) {
                fare = ((NightService) cab).addNightSurcharge(fare);
            }
            total += fare;
            System.out.printf(Locale.US, "%s: %.2f%n", type, fare);
        }
        System.out.printf(Locale.US, "Total: %.2f%n", total);
        sc.close();
    }
}
```

Sample input

```
4
MINI 8 DAY
SEDAN 10 NIGHT
SUV 20 DAY
MINI 5 NIGHT
```

Expected output

```
MINI: 100.00
SEDAN: 168.00
SUV: 360.00
MINI: night service not available
Total: 628.00
```

Problem 5 – Home Appliance Energy Report

OOP design / explanation

Appliance contains the shared energy and cost calculations. SaverMode is a capability interface implemented only by AC and washer. If SAVER is requested for a fridge or TV, the program reports it and excludes that unsupported request from the total.

Complete Java solution

```
import java.util.Locale;
import java.util.Scanner;

interface SaverMode {
    double applySaver(double units);
}

abstract class Appliance {
    protected abstract double getPowerWatts();

    public final double calculateUnits(double hours) {
        return getPowerWatts() * hours / 1000.0;
    }
    public final double calculateCost(double units) {
        return units * 8.0;
    }
}

class Fridge extends Appliance {
    protected double getPowerWatts() { return 150.0; }
}
class AirConditioner extends Appliance implements SaverMode {
    protected double getPowerWatts() { return 1500.0; }
    public double applySaver(double units) { return units * 0.75; }
}
class Television extends Appliance {
    protected double getPowerWatts() { return 100.0; }
}
class WashingMachine extends Appliance implements SaverMode {
    protected double getPowerWatts() { return 500.0; }
    public double applySaver(double units) { return units * 0.75; }
}

public class HomeApplianceEnergyReport {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();
        double totalCost = 0.0;

        for (int i = 0; i < n; i++) {
            String line = sc.nextLine().trim();

            String[] parts = line.split("\\\\s+");
            String type = parts[0].toUpperCase(Locale.ROOT);
            double hours = Double.parseDouble(parts[1]);
            boolean saverRequested = parts.length > 2
                    && parts[2].equalsIgnoreCase("SAVER");
            Appliance appliance;

            if (type.equals("FRIDGE")) {
                appliance = new Fridge();
            } else if (type.equals("AC")) {
                appliance = new AirConditioner();
            } else if (type.equals("TV")) {
                appliance = new Television();
            } else if (type.equals("WASHER")) {
                appliance = new WashingMachine();
            } else {
                throw new IllegalArgumentException("Unknown appliance: " + type);
            }

            if (saverRequested && !(appliance instanceof SaverMode)) {
                System.out.println(type + ": saver mode not supported");
                continue;
            }

            double units = appliance.calculateUnits(hours);
            if (saverRequested) {
                units = ((SaverMode) appliance).applySaver(units);
            }
            double cost = appliance.calculateCost(units);
            totalCost += cost;
            System.out.printf(Locale.US,
                    "%s: Units=%.2f Cost=%.2f%n", type, units, cost);
        }
        System.out.printf(Locale.US, "Total Cost: %.2f%n", totalCost);
        sc.close();
    }
}
```

Sample input

```
4
FRIDGE 24
```

```
AC 8 SAVER
TV 5
WASHER 2 SAVER
```

Expected output

```
FRIDGE: Units=3.60 Cost=28.80
AC: Units=9.00 Cost=72.00
TV: Units=0.50 Cost=4.00
WASHER: Units=0.75 Cost=6.00
Total Cost: 110.80
```